DLDCA Outlab: Dead Man In Rushville

Week 10

November 1, 2026

"In brightest day, in blackest night,
no dead block shall escape my sight."

Introduction

Welcome back to Simulator World! Over the last two weeks you learnt how ChampSim moves a request through the cache hierarchy, wrote your own replacement policies at the L2C and bypassed LLC fills in the Week 10 inlab. In this outlab you move down to the LLC and deal with its biggest waste: *dead blocks*. A dead block sits in the cache and is never used again before it is evicted. You will measure how many there are, predict them from the instruction that brought them in, and use that prediction to keep them out of the LLC.

Grading Policy

This outlab is graded. The deadline will be announced on Piazza. The AI policy is strict. Do not use AI agents to write code or run experiments on your behalf. You will notice that the ChampSim codebase is more than just a few files with many moving parts. Attempting this exercise on your own will help you understand and work with larger codebases in general (an important skill outside of the course as well) and might be more efficient than prompting an AI for it. Feel free to discuss doubts on Piazza, without revealing answers of course. We will be running AI plagiarism checks as well.

There are 5 tasks.

1. **SRRIP at the LLC.** Replace LRU at the LLC with SRRIP, a policy that predicts how soon a block will be used again.
2. **A dead block predictor.** Build a predictor that learns, per PC, whether blocks are likely to die, and count how many LLC blocks are actually dead.
3. **Bypassing dead blocks.** Use the predictor to skip storing blocks in the LLC that it calls dead. This is only an LLC bypass; the LLC does *not* become exclusive.
4. **Evicting dead blocks first.** Make SRRIP pick a block predicted dead as its victim before anything else.
5. **Tuning the predictor.** Change the predictor's counter width and threshold, measure the IPC and explain the trend. This task has no code, only experiments and answers.

Every task is code plus written answers

Each task folder has the file(s) you complete **and** a `questions.txt` (Task 5 has only the questions). Write your answers in `questions.txt`, below each question, and submit it with your code. Keep the questions in the file and roughly the given layout (the question, then your answer under `Answer:`), so that we can find your answers quickly. We grade your code with `make longcheck` and read every answer. The answers are meant to help you solidify your understanding, please avoid using AI for them.

Keep the Week 9 and Week 10 PDFs and slides close. This PDF repeats only what you need.

Setting up

Where to run. Any Unix system works: native Linux, macOS, WSL on Windows, or a Linux virtual machine. The lab machines have everything installed. If your own machine gets in the way, use the lab machines.

Step 1: Check your tools. You need `g++` (with C++17), `make`, `python3` and `xz`. Each of these commands should print a version, not `command not found`:

```
$ g++ --version
$ make --version
$ python3 --version
$ xz --version
```

If one is missing, install it. On Ubuntu or WSL, run `sudo apt install build-essential python3 xz-utils`. On macOS, run `xcode-select -install` for `g++` and `make`, and `brew install xz` for `xz`.

Step 2: Get the traces. We use the same four traces as in Weeks 9 and 10: `soplex`, `bwaves`, `mcf` and `gcc`. On the lab machines they are already in `/opt/CS231_Traces`, so skip this step. On your own machine, download these four files into one folder from our Google Drive folder (`https://drive.google.com/drive/folders/1MZnn0J9gbPvjEUzumMlLouagoTv1Ou2Y?usp=sharing`). They are also on the DPC-3 page (`https://dpc3.compas.cs.stonybrook.edu/champsim-traces/speccpu/`).

```
603.bwaves_s-891B.champsimtrace.xz
605.mcf_s-994B.champsimtrace.xz
602.gcc_s-2226B.champsimtrace.xz
soplex_66B.trace.xz
```

Step 3: Tell the makefile where the traces are. Run this in every new terminal, or add it to your `~/.bashrc`:

```
$ export CS231_TRACE_DIR=/path/to/your/trace/folder
$ ls $CS231_TRACE_DIR
602.gcc_s-2226B.champsimtrace.xz    605.mcf_s-994B.champsimtrace.xz
603.bwaves_s-891B.champsimtrace.xz  soplex_66B.trace.xz
```

The makefile finds a trace by name: `trace=soplex` picks the file matching `*soplex*.xz` in `CS231_TRACE_DIR`. If the variable is not set, it looks in `/opt/CS231_Traces`.

Step 4: Check that everything works. From the lab folder, build the given ChampSim with LRU at the LLC and run it once. This takes about a minute:

```
$ make lru
Building .lru_champsim/bin/champsim from ...
.lru_champsim/bin/champsim --warmup_instructions 5000000 ...
CPU 0 cumulative IPC: ...
...
LLC MPKI (LOAD+RFO): ...
```

If you see the IPC line, you are ready to start.

We use a modified ChampSim

The `Champsim/` folder in this lab is **not** the stock ChampSim from Weeks 9 and 10. It has a few additions for this lab, listed at the end of the next section. Do not edit it; the makefile builds every task from it. You edit only the files already in the task folders.

Running ChampSim by hand

This is the raw way to build and run ChampSim, the same as in Week 9. From inside a ChampSim folder: pick the policies in `champsim_config.json`, configure (reads the JSON and writes the `Makefile`), compile (the binary goes to `bin/`), then run it on a trace with two instruction counts.

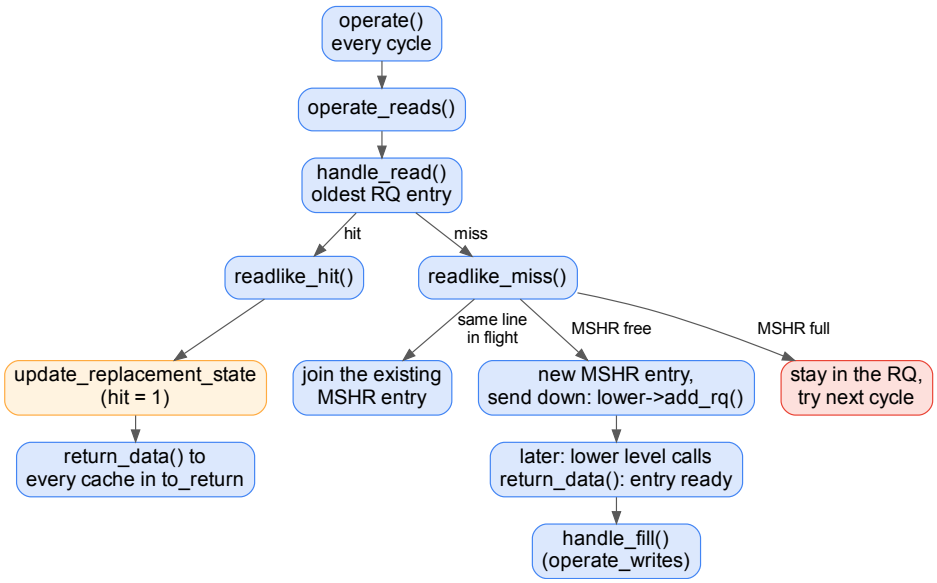
```
$ make clean                      # only if you built here before
$ vim champsim_config.json        # e.g. the LLC replacement policy
$ ./config.sh champsim_config.json
$ make
$ ./bin/champsim --warmup_instructions 5000000 \
                 --simulation_instructions 10000000 \
                 $CS231_TRACE_DIR/soplex_66B.trace.xz
```

Changed the JSON or added a policy? Rerun `./config.sh` and `make`. `make` alone does not read the JSON.
You do not need to do this for the tasks. `make taskN` does all of these steps for you, in a private copy of `Champsim/` (see Building and testing).

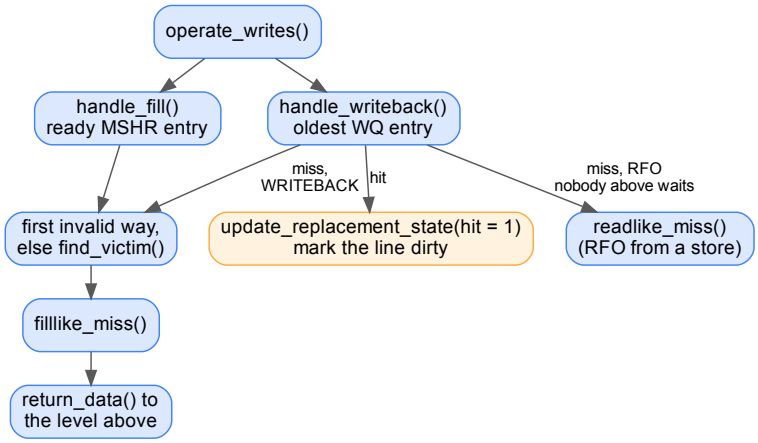
A map of `cache.cc`

What ChampSim already does

The same `CACHE` code runs for every cache: `L1I`, `L1D`, `cpu0_L2C`, `LLC` and the TLBs. `NAME` tells you which one is running. In every cycle ChampSim calls `operate()` on each cache, which first handles fills and writebacks (`operate_writes`) and then reads (`operate_reads`). A cache has a read queue (`RQ`), a write queue (`WQ`), a prefetch queue and an MSHR, which holds misses that wait for data from the level below.



The read path. A read either hits, joins an MSHR entry already waiting for the same block, or gets a new MSHR entry and goes down.



The write path. Fills come from ready MSHR entries and writebacks from the WQ. Both pick a way and store the block through `filllike_miss`.

These are the functions you will read and edit:

`get_set`, `get_way` *Lookup.* `get_set` takes the set index from the address. `get_way` searches that set and returns the matching way, or `NUM_WAY` on a miss.

`handle_read` Takes the oldest packet in the `RQ` and calls `readlike_hit` on a hit or `readlike_miss` on a miss.

`readlike_hit` Copies the block's data into the packet, calls `update_replacement_state` with `hit = 1`, counts the hit and returns the data to every cache waiting for it (`to_return`).

`readlike_miss` *MSHR handling.* If the block is already in the MSHR, the request joins that entry. Otherwise it allocates an entry and sends the request down with `lower_level->add_rq`. If the MSHR or the lower queue is full, it returns `false` and the request tries again next cycle.

`handle_writeback` Takes the oldest packet in the `WQ`. On a hit it calls `update_replacement_state` with `hit = 1` and `type = WRITEBACK` and marks the block dirty. On a miss it picks a way (an invalid way, else `find_victim`) and calls `filllike_miss`.

`handle_fill` Runs when the data for an MSHR entry has arrived. It picks a way (an invalid way, else `find_victim`), calls `filllike_miss` to store the block, *returns the data* to every cache in `to_return`, and frees the MSHR entry.

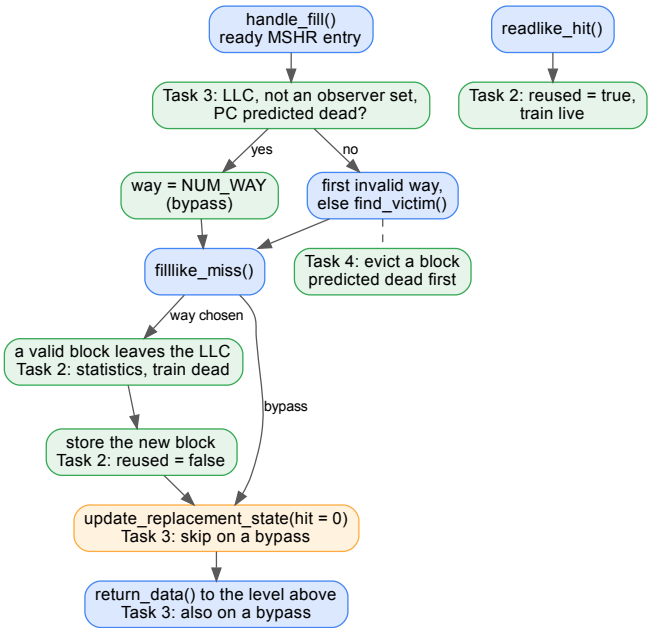
`filllike_miss` *Insertion.* If the victim is dirty, it writes the victim back to the level below. Then it overwrites the block with the new one, calls `update_replacement_state` with `hit = 0` and counts the miss. It treats `way == NUM_WAY` as a *bypass* and stores nothing.

A replacement policy plugs in through two functions. ChampSim calls `find_victim` when a set is full, and `update_replacement_state` after every hit (`hit = 1`) and every fill (`hit = 0`). The `type` argument is `LOAD`, `RFO`, `PREFETCH`, `WRITEBACK` or `TRANSLATION`.

What we added for this lab

1. The dead block predictor, `inc/sampler.h`, which you complete in Task 2.
2. The dead block statistics, printed by `print_dead_stats()`.
3. Writeback packets carry the PC of their block instead of 0.

The figure below shows where each task changes the LLC. Green boxes are code you add.



Where Tasks 2, 3 and 4 hook into the LLC.

The numbers we check

IPC, LLC MPKI and the dead block statistics

- **IPC**, from the line `CPU 0 cumulative IPC`. Higher is better.
- **LLC MPKI**: LLC demand misses per 1000 instructions. This week it counts only `LOAD` and `RFO` misses:

$$\text{LLC MPKI} = \frac{\text{LLC LOAD MISS} + \text{LLC RFO MISS}}{\text{simulation instructions}/1000}$$

  We leave writeback misses out because of Task 3. A block that skipped the LLC can come back later as an L2C writeback that misses in the LLC, and the core never waits for that miss.
- **LLC DEAD ON ARRIVAL** and **LLC PREDICTED DEAD**: the two percentages you compute in Task 2. They read 0.00% until then.

The lab makefile prints all of them, and `make quickcheck` and `make longcheck` compare all four with ours.

Building and testing

The lab makefile (run it from the lab folder, not from `Champsim/`)

`make task1`	build with your `task1/srrip_llc.cc` at the LLC; quick run on `soplex`
`make task2`	the same with `task1/srrip_llc.cc`, `task2/sampler.h`, `task2/cache.cc`
`make task3`	the same with `task1/srrip_llc.cc`, `task2/sampler.h`, `task3/cache.cc`
`make task4`	the same with `task4/srrip_llc.cc`, `task2/sampler.h`, `task3/cache.cc`
`make task5`	your Task 4 code with `COUNTER_BITS=[b] THRESHOLD=[t]` (see Task 5)
`make lru`	the given ChampSim with `lru` at the LLC: the baseline
`... trace=mcf`	use another trace: `soplex`, `bwaves`, `mcf` or `gcc`
`... long=1`	long run (20M + 25M) instead of quick (5M + 10M); only needed for `make lru` and `make task5`
`make quickcheck`	sanity check: every task, quick runs (5M + 10M), against our numbers
`make longcheck`	grading check: every task, long runs (20M + 25M), against our numbers
`make quickcheck-task2`	one task (also `longcheck-task2`, and the same for the other tasks)
`make diff-task3`	print your `task3/cache.cc` against the given `Champsim/src/cache.cc`
`make clean`	delete everything that was built

`make taskN` copies `Champsim/` to a hidden folder `.taskN_champsim/`, copies that task's files into it, sets `srrip_llc` as the LLC policy and builds it. The copy is made fresh on every build, so edit only the files in the task folders. Tasks 3 and 4 reuse files from earlier tasks, so a mistake in `task2/sampler.h` also fails Tasks 3 and 4. Get each task passing before you start the next one.

There are two checks. Both build each task once, run it on all four traces at the same time, and compare the IPC, LLC MPKI and both dead block percentages with ours. ChampSim is

deterministic, so a correct solution gives exactly our numbers.

Run `make quickcheck` (against `taskN/expected_output.txt`, about two minutes per task) while you work. **We grade with** `make longcheck` (against `taskN/expected_output_long.txt`, about five minutes per task), so run it before you submit.

If any value differs, the check prints yours and ours:

```
$ make quickcheck-task1
Building .task1_champsim/bin/champsim from task1/srrip_llc.cc
Champsim/inc/sampler.h Champsim/src/cache.cc
[task1] soplex  | FAIL (differs from the reference)
    got:      0.434452 17.9160 0.00% 0.00%
    expected: 0.366666 22.9727 0.00% 0.00%
...
```

The four values are IPC, LLC MPKI, dead on arrival and predicted dead. The traces run in parallel, so the lines can come in any order. If a check keeps failing for no reason you can see, run `make clean` and try again.

Structure of submission/templates

Edit only the files marked `(TODO)`.

```
your-roll-no/
|- task1/
    |- srrip_llc.cc (TODO)
    |- questions.txt (TODO)
    |- expected_output.txt
    |- expected_output_long.txt
|- task2/
    |- sampler.h (TODO)
    |- cache.cc (TODO)
    |- questions.txt (TODO)
    |- expected_output.txt
    |- expected_output_long.txt
|- task3/
    |- cache.cc (TODO)
    |- questions.txt (TODO)
    |- expected_output.txt
    |- expected_output_long.txt
|- task4/
    |- srrip_llc.cc (TODO)
    |- questions.txt (TODO)
    |- expected_output.txt
    |- expected_output_long.txt
|- task5/
    |- questions.txt (TODO)
```

Package your work with the given script. It asks for your roll number and writes `your-roll-no.tar.gz`:

```
$ ./build_archive.sh
Enter Student Roll Number: your-roll-no
Packaging assignment files...
  [+] Copied: ...
Successfully created your-roll-no.tar.gz
```

Upload `your-roll-no.tar.gz` on Moodle under Week 10 Outlab.

Tasks

Task 1: SRRIP at the LLC

Every block that the L2C misses on lands in the LLC, so the LLC's replacement policy decides which of those blocks get a second chance. Right now that policy is LRU. LRU keeps an exact recency order in each set, puts every new block at the most recently used position, and evicts the least recently used block. A block that is used once and never again therefore gets the full trip from the top of the order to the bottom before it leaves.

SRRIP (Static Re-Reference Interval Prediction, Jaleel et al., ISCA 2010) takes a different view. It does not track an exact order. It keeps a small guess per block of *how soon the block will be used again*, and a new block starts out with a pessimistic guess. A block earns a better guess only by getting a hit.

The RRPV. Each block has a re-reference prediction value (RRPV), a 2-bit number from 0 to maxRRPV = 3. An RRPV of 0 means "used again soon" and 3 means "not for a long time". SRRIP is three rules on this number:

```
insertion (fill):  RRPV[new block] = maxRRPV - 1    # long, not the longest
promotion (hit):   RRPV[block] = 0                  # reused: keep it
victim selection:
    loop:
        if some block in the set has RRPV == maxRRPV:
            return the first such way
        for every block in the set: RRPV = RRPV + 1  # age the set
```

When nobody in the set is at 3, SRRIP ages the whole set by one and looks again. Since every RRPV goes up together, the order between blocks stays the same, and the block with the highest RRPV is evicted first.

Example. Here is one 4-way set holding `A`, `B`, `C` and `D`, with each block's RRPV after the colon:

step	way 0	way 1	way 2	way 3	what happens
start	A:0	B:1	C:2	D:2	
miss on E	A:1	B:2	C:3	D:3	no block at 3, so age every block once
	A:1	B:2	E:2	D:3	C is the first block at 3: evict it, insert E at 2
hit on B	A:1	B:0	E:2	D:3	promote B
miss on F	A:1	B:0	E:2	F:2	D is at 3: evict it, insert F at 2

Notice that E and F enter at 2, one step from eviction, while B, which just got a hit, is safe at 0 for a while.

Writebacks. At the LLC, `update_replacement_state` is also called for writebacks from the L2C (`type == WRITEBACK`). The rule for them is: **a writeback hit is not a reuse, and a writeback fill has the lowest priority.** Write it as code yourself (see Q2 in `questions.txt`).

What to do. Complete `task1/srrip_llc.cc`. First define a data structure that holds one RRPV per block; see the Week 10 `labexam` for the idea.

1. `initialize_replacement()`: create your structure and give every block the RRPV `maxRRPV`.
2. `update_replacement_state(..., set, way, ..., type, hit)`: insertion when `hit == 0`, promotion when `hit == 1`, and the writeback rule above.
3. `find_victim(..., set, ...) -> way`: victim selection as in the pseudocode. Scan from way 0 and return the first way whose RRPV is `maxRRPV`.
4. Run `make task1` and then `make quickcheck-task1`. Before you submit, run `make longcheck-task1`.

Reference numbers

Long runs give these numbers. `make longcheck-task1` prints yours for every trace; the LRU columns come from `make lru long=1 trace=[trace]`. **Your IPC and LLC MPKI must match the SRRIP columns.**

trace	IPC (LRU)	LLC MPKI (LRU)	IPC (SRRIP)	LLC MPKI (SRRIP)
`soplex`	0.323267	27.6768	0.348748	25.0006
`bwaves`	0.243644	26.1783	0.243545	26.1766
`mcf`	0.409243	10.1766	0.40079	10.9336
`gcc`	0.113813	69.9676	0.113819	69.9674

Questions. Answer the questions in `task1/questions.txt`.

Task 2: A PC-Based Dead Block Predictor

A block is *dead* from its last use until it is evicted. If a block is evicted without a single hit since it was filled, it was *dead on arrival*: storing it gained nothing, and it pushed out a block that might have been used again. The LLC is full of such blocks. On `soplex`, about 9 out of every 10 blocks the LLC evicts were never hit.

If we could tell, at the moment a block arrives, that it will be dead, we could avoid storing it (Task 3) or evict it early (Task 4). In this task you build the part that makes that guess, and you measure how many LLC blocks are really dead. The design is a simplified version of the sampling dead block predictor (Khan et al., MICRO 2010).

The idea: the PC predicts deadness. The instruction that brought a block in says a lot about the block's future. If the load at PC `x` has mostly brought in blocks that died, the next block it brings in will probably die too. So we keep a confidence counter per PC. It goes up when a block brought in by `x` is evicted without a hit, and down when such a block gets a hit. When the counter is high, we predict that blocks from `x` are dead.

Each LLC block already remembers the PC that brought it in, in `block.ip`. We use the lower 14 bits of that PC as the block's *signature*; `signature(ip)` in `sampler.h` computes it for you.

Three skewed tables. We cannot afford one counter for every possible signature, so signatures share counters, and two signatures can map to the same counter. If `x` brings in dead blocks and `y` brings in live ones, a counter they share is wrong for one of them. This is called *destructive interference*. To make it rare, we use three tables of 4096 counters each, and each table has its own hash function from signature to index. Two signatures are unlikely to collide in all three tables at once.

```
sig s --hash1--> counters[0][hash1(s)] --+
      --hash2--> counters[1][hash2(s)] --+--> sum (0..9) > 8 ? dead : live
      --hash3--> counters[2][hash3(s)] --+
```

The predictor works like this:

- Each counter is a 2-bit saturating counter. It stays between 0 and 3 (`DBP_MAX_COUNT`).
- `predict_to_be_dead(s)` adds the three counters that `s` maps to and returns `true` if the sum is greater than `DBP_THRESHOLD` (8).
- `update(s, dead)` adds 1 to each of the three counters if `dead` is `true`, and subtracts 1 otherwise. A counter never goes below 0 or above 3.

Learning from a few sets. Real hardware cannot afford to update the predictor on every LLC access. Watching a small sample of sets is enough to learn how each PC behaves, so the predictor learns only from every 64th set, 32 of the 2048 LLC sets. We call them the *observer sets*, and `is_observer(set)` tells you whether a set is one. The predictor is trained on two events, both only in observer sets:

- On a read hit in the LLC (`readlike_hit`), call `update` with the block's signature and `dead = false`.
- When a valid block that was never reused is evicted from the LLC, call `update` with its signature and `dead = true`.

Training happens always, also during warmup. Only the statistics below wait for the end of warmup.

Measuring deadness. To know whether a block was reused, you keep a flag per LLC block correct: set it to `false` when a block is filled and to `true` when it gets a read hit. Then, for every valid block evicted from the LLC, in all sets and **only after warmup** (`warmup_complete[handle_pkt.cpu]`), update these counters in `cache.cc`:

- `llc_evictions` counts every such eviction.
- `llc_dead_evictions` counts the evicted blocks that were never reused.
- `llc_predicted_dead` counts the evicted blocks that the predictor calls dead at that moment.

`print_dead_stats()` is given. It prints the last two as percentages of the first, as `LLC DEAD ON ARRIVAL` and `LLC PREDICTED DEAD`.

What to do. You complete two files, and writing the relevant data structures is part of the task: the counter tables of the predictor and the reuse flag of the LLC blocks.

1. In `task2/sampler.h`, declare and initialize the counter tables, then write `predict_to_be_dead(sig) -> bool` and `update(sig, dead)`. The sizes are given as macros (`DBP_TABLES`, `DBP_ENTRIES`); the hash functions and observer sets are given too, do not change them. Use the macros `DBP_MAX_COUNT` and `DBP_THRESHOLD`, not the numbers 3 and 8: Task 5 changes them.
2. In `task2/cache.cc`, define your reuse structure at the top and fill the `// TODO N Task 2` places. In `readlike_hit` you mark the block reused and train on the hit. In `filllike_miss` you count and train on the evicted block, then reset the flag for the new block. Every change applies to the LLC only. **Hint: Use `if (NAME == "LLC")` to run a branch of code only for LLC.**
3. Run `make task2` and then `make quickcheck-task2`. Before you submit, run `make longcheck-task2`.

The predictor does not change any decision yet, so the IPC and LLC MPKI of Task 2 are the same as in Task 1. Only the two percentages are new.

Reference numbers

Long runs give these percentages; `make longcheck-task2` prints yours for every trace. **Your two percentages must match.**

trace	dead on arrival (%)	predicted dead (%)
`soplex`	89.57	83.12
`bwaves`	100.00	99.97
`mcf`	64.33	78.44
`gcc`	99.99	99.98

Questions. Answer the questions in `task2/questions.txt`.

Task 3: Bypassing Dead Blocks at the LLC

Now that the predictor can call a block dead, let us act on it. If a block is going to be evicted without a hit anyway, storing it in the LLC only pushes out another block that might have been reused. So when DRAM returns a block whose PC is predicted dead, the LLC sends the data up to the L2C and does not store the block. This is called a *bypass*.

We only implement LLC bypass, not a fully exclusive LLC. You saw bypassing in the Week 10 inlab, where it kept the LLC exclusive of the L2C. Here the LLC stays non-inclusive as it was: every block that is not predicted dead is stored in the LLC as before, and we skip only the blocks that the predictor calls dead.

Keep the observer sets learning. The predictor learns only from hits and evictions in the observer sets. If an observer set bypassed a PC that is predicted dead, no block from that PC would enter it again, and the predictor could never find out that the PC has become live. **Observer sets never bypass.**

What to do.

1. Copy your Task 2 code into `task3/cache.cc`, at the same places as in `task2/cache.cc`.
2. In `handle_fill`, at `// TODO 1 Task 3: bypass`, implement bypass logic when the cache is the LLC (hint: `if (NAME == "LLC")` runs a branch only in the LLC), the set is not an observer set and the predictor calls the arriving block dead. Figure out where the arriving block's PC can be found.
3. In `handle_fill`, at `// TODO 2 Task 3: return data`, return the data for a bypass too.
4. In `filllike_miss`, at `// TODO 3 Task 3: skip replacement update`, skip the replacement update for a bypass.
5. Run `make task3` and then `make quickcheck-task3`. Before you submit, run `make longcheck-task3`.

Reference numbers

Long runs give these numbers; `make longcheck-task3` prints yours for every trace. **Your IPC and LLC MPKI must match the Task 3 columns.**

trace	IPC (Task 1)	LLC MPKI (Task 1)	IPC (Task 3)	LLC MPKI (Task 3)
`soplex`	0.348748	25.0006	0.417835	19.1989
`bwaves`	0.243545	26.1766	0.244678	26.1766
`mcf`	0.40079	10.9336	0.381294	12.9796
`gcc`	0.113819	69.9674	0.115012	68.9636

Questions. Answer the questions in `task3/questions.txt`. `make diff-task3` prints the diff that Q1 asks for.

Task 4: Evicting Dead Blocks First

Task 3 catches dead blocks at the door, but only the ones whose PC already looks dead when they arrive. The predictor keeps learning, so a block filled while its PC looked live may belong to a PC that is predicted dead a little later. SRRIP knows nothing about this and keeps the block until its RRPV climbs to `maxRRPV`. In this task SRRIP asks the predictor first.

The new victim selection. Before the SRRIP loop, scan the set from way 0 for a block whose PC is now predicted dead. If you find one, evict it. If not, fall back to SRRIP victim selection from Task 1. As in Task 3, **observer sets stay plain SRRIP**, so that the predictor keeps learning from them.

What to do.

1. Copy your Task 1 code into `task4/srrip_llc.cc`.
2. Fill `// TODO 1 Task 4: evict dead blocks first, then do the SRRIP loop` in `find_victim`.
3. Run `make task4` and then `make quickcheck-task4`. Before you submit, run `make longcheck-task4`.

Reference numbers

Long runs give these numbers; `make longcheck-task4` prints yours for every trace. **Your IPC and LLC MPKI must match the Task 4 columns.**

trace	IPC (Task 3)	LLC MPKI (Task 3)	IPC (Task 4)	LLC MPKI (Task 4)
`soplex`	0.417835	19.1989	0.42207	18.9974
`bwaves`	0.244678	26.1766	0.244826	26.1766
`mcf`	0.381294	12.9796	0.379067	13.1640
`gcc`	0.115012	68.9636	0.115021	68.9666

Questions. Answer the questions in `task4/questions.txt`.

Task 5: Tuning the Predictor

The predictor from Task 2 has two numbers we picked for you: each counter is 2 bits wide, and a PC is dead when the three counters add up to more than 8. In this task you change them, measure what happens, and explain it. There is no code to write; you only run experiments.

What the two knobs do. With `COUNTER_BITS` $= b$, each counter saturates at $\text{MAX} = 2^b - 1$, so the sum of the three counters goes from 0 to $3 \cdot \text{MAX}$. `THRESHOLD` decides how far up that range the sum must climb before a PC counts as dead; the highest useful value is $3 \cdot \text{MAX} - 1$.

How to run it. `make task5` builds the same files as `make task4`, with the two values you pass, and runs it like `make taskN`:

```
$ make task5 long=1 trace=mcf COUNTER_BITS=3 THRESHOLD=20
```

Your Task 2 to 4 files stay at the defaults, so `make longcheck` is not affected.

What to do.

1. Run long runs of `soplex` and `mcf` for these six settings of (`COUNTER_BITS`, `THRESHOLD`):

   $$(2,2),\ (2,8),\ (3,6),\ (3,20),\ (4,14),\ (4,44)$$

   For each width, the first threshold is low ($\text{MAX} - 1$) and the second is the highest possible ($3 \cdot \text{MAX} - 1$). $(2,8)$ is the setting of Tasks 2 to 4. That is 12 long runs; each takes a few minutes.

2. Fill the IPC table in `task5/questions.txt`.

3. Answer Q2 in one or two lines. Be precise: name the fastest setting for each trace, describe how IPC changes with the threshold and with the width, and explain why using what you know about each trace from Tasks 2 and 3.

Questions. Answer the questions in `task5/questions.txt`.